

BaseML功能详解

我们的传统机器学习（Machine Learning）有很多算法，但总的来说，可以分为三大类：分类、回归和聚类。BaseML和sklearn不同之处，也就体现于此，sklearn尽管在拟合、预测等函数上对各模块做了统一，但并没有明确指出这样的三大类划分方式。这三类也有着特有的数据输入格式。

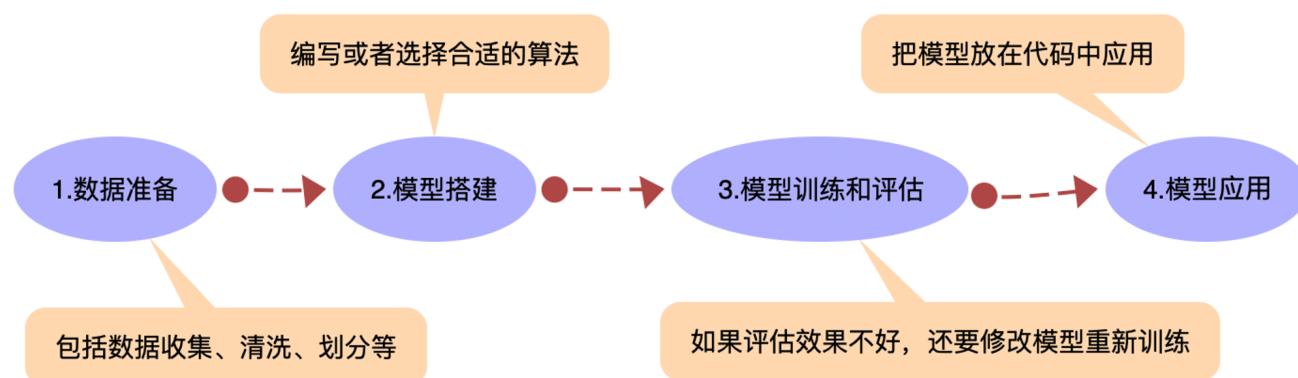
文档涉及的部分代码见XEdu帮助文档配套项目集：

[https://www.openinnolab.org.cn/pjlab/project?](https://www.openinnolab.org.cn/pjlab/project?id=64f54348e71e656a521b0cb5&sc=645caab8a8efa334b3f0eb24#public)

[id=64f54348e71e656a521b0cb5&sc=645caab8a8efa334b3f0eb24#public](https://www.openinnolab.org.cn/pjlab/project?id=64f54348e71e656a521b0cb5&sc=645caab8a8efa334b3f0eb24#public)

机器学习的基本流程

机器学习实际上分为两个阶段，首先是模型训练过程，即“学习”；然后是模型推理过程，即“应用”。典型的机器学习流程可以分为数据准备、模型搭建、模型训练与评估、模型应用等环节（如下图）。



1.数据准备

数据是描述客观事物或现象的符号记录，可以是数字、文字、图像、声音等形式。机器学习需要很多数据，我们称之为“数据集”。要训练怎样的模型，就要准备怎样的数据。例如要训练温度转换的模型，就要准备很多条类似“摄氏温度和华氏温度对应表”的数据。

2.模型搭建

搭建机器学习的模型，核心工作是实现一个具有特定功能的算法。实现机器学习的算法需要编写程序，难度较大。但好消息是Python有多个机器学习的库，这些库中内置了各种优秀的算法，只要根据需求选择合适的算法，就可以直接完成模型的搭建。

3.模型训练与评估

对于训练好的模型，需要评估一下其推理能力，类似人学习了某个课程后，还要做点单元小测试，看看掌握了多少。对回归任务来说，简单好用的评估指标之一是R平方值，对分类任务来说，一般选择准确率。通过比对推断结果与实际标注结果的差异，可以计算出评估指标。如果推理效果不好，要重新检查数据和模型，再次训练。

4.模型应用

当训练出来的模型的评估表现不错，那就可以保存模型。保存出来的模型文件，可以导入使用或供其他程序使用。其实模型应用环节和传统编程就差别不大了，只要输入一组新数据，就能输出预测结果。

机器学习典型算法一览表

点击对应的 回归 或 分类，可连接至该算法更详细的说明文档，向右拖动可查看各算法对应的常用参数及其默认值。

算法名称	适合任务	典型任务
线性回归 (LinearRegression)	回归	适用于预测房价、预测销售额、贷
多项式回归算法 (Polynomial)	回归	适用于预测房价、预测销售额、贷
最近邻分类 (KNN, K-Nearest Neighbors)	分类	识别数字、判断邮件是否为垃圾邮
支持向量机 (SVM, Support Vector Machine)	分类/回归	文本分类、图像识别、股票市场分
决策树算法 (CART)	分类/回归	适用于客户分级、疾病诊断等。
随机森林算法 (RandomForest)	分类/回归	信用评分、医疗分析、股票市场行
自适应增强算法 (AdaBoost)	分类/回归	人脸识别、客户流失预测、分类任
多层感知机算法 (MLP)	分类/回归	适用于语音识别、手写识别、自然
.....		

更多算法详见[后文](#)。

示例代码

核心代码（训练）：

```
from BaseML import Regression as reg # 从库文件中导入回归任务模块
model = reg('LinearRegression') # 实例化线性回归模型
model.set_para(fit_intercept=True) # 设定模型参数
model.load_tab_data('./data_train.csv') # 载入训练数据
model.train() # 训练模型
model.valid('./data_val.csv', metrics='r2') # 载入验证数据并验证
model.save('./mymodel.pkl') # 保存模型供应用
```

其中 `设定模型参数` 非必要，各参数都有默认值，具体见[上表](#)。

核心代码（推理）：

```
from BaseML import Regression as reg # 从库文件中导入回归任务模块
model = reg('LinearRegression') # 实例化线性回归模型
model.load('./mymodel.pkl')
data = [[-1, 52, 38]] # 指定一组新数据，根据训练模型时使用的数据来定
result = model.inference(data=data) # 进行模型推理
print(result)
```

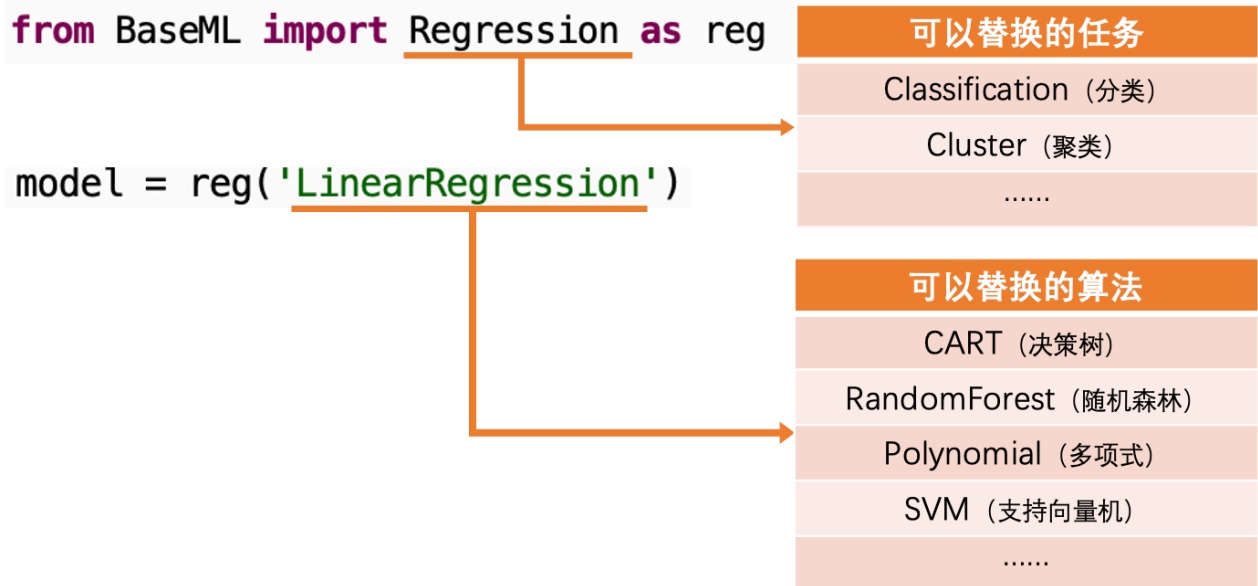
BaseML代码详解

1. 导入包与搭建模型

库文件的导入只需要一行代码，根据机器学习的任务导入相应的库。“Regression”模块内置了回归任务的常见算法，“Classification”模块内置了分类任务的常见算法，“Cluster”模块则内置了聚类任务的常见算法。

```
from BaseML import Regression as reg # 从库文件中导入回归任务模块
model = reg('LinearRegression') # 实例化线性回归模型
model.set_para(fit_intercept=True) # 设定模型参数
```

在BaseML中使用不同算法：



第二句代码属于模型搭建，除了实例化模型，选择部分算法如需额外设置参数可添加参数设置的代码（涉及的参数见上文的机器学习典型算法一览表），一般有如下三种搭建形式，下面以搭建多层感知机算法为例进行说明，此算法一般需要设置隐藏层“hidden_layer_sizes”，hidden_layer_sizes:(100,200)表示2层神经元数量为100和200的隐藏层。如下是它的三种搭建形式。

```
# 第一种形式
param = {"hidden_layer_sizes":(100,200)}
model = reg("MLP",para=param)

# 第二种形式
model = reg("MLP")
model.para = {"hidden_layer_sizes":(100,200), "activation":'relu', "solver":'adam'}

# 第三种形式
model = reg("MLP")
model.set_para(hidden_layer_sizes=(100,200), activation='relu', solver='adam')
```

拓展功能：查看各任务拥有的算法以及类注释

```
from BaseML import Regression as reg
print(reg.__doc__)
# BaseML中的回归模块, 包含['LinearRegression'(线性回归), 'CART'(决策树回归), 'RandomForest'(随机森林回归), 'Polynomial'(多项式回归), 'Lasso'(角回归), 'Ridge'(岭回归), 'SVM'(支持向量机回归), 'AdaBoost'(自适应增强回归), 'MLP'(多层感知机回归)]回归算法.

from BaseML import Classification as cls
print(cls.__doc__)
# BaseML中的分类模块, 包含['KNN'(K近邻分类), 'SVM'(支持向量机分类), 'NaiveBayes'(朴素贝叶斯分类), 'CART'(决策树分类), 'AdaBoost'(自适应增强分类), 'MLP'(多层感知机分类), 'RandomForest'(随机森林分类)]分类算法.

from BaseML import Cluster as clt
print(clt.__doc__)
# BaseML中的聚类模块, 包含['Kmeans'(K均值聚类), 'Spectral clustering'(谱聚类), 'Agglomerative clustering'(层次聚类), 'Birch'(二叉可伸缩聚类树聚类)]聚类算法.
```

2. 数据载入

BaseML库支持各种形式载入数据。可以使用内置函数载入数据，也可以使用外部工具读入数据后，赋值给BaseML的对象。

使用内置函数载入数据

(1) 针对表格类型数据

表格类型数据一般使用CSV格式。核心方法有 `load_tab_data` 和 `load_dataset` 两种。其中 `load_tab_data` 适合特定的单标签数据，`load_dataset` 则适用绝大多数的数据。

方法1：使用 `load_tab_data` 方法直接载入一个特定CSV文件

`load_tab_data` 方法对CSV文件有严格的格式要求：

- 数据文件每行一条记录（首行为表头，数据从第 2 行开始），输入数据（特征）列在前，输出数据（目标或标签）列在后，即最后一列是输出数据（标签），其余列都是输入数据（特征）。
- 数据类型为数值类型且无缺失值。

数据格式要求如下：

	A	B	输入数据	D	E	输出数据
1	feature1	feature2	feature3	feature4	feature5	Label
2	-1	52	36	59	6.5	155
3	1	70	41	65	8.5	170
4	-1	68	39	64	8.3	168
5	-1	56	38	61	7	160
6	1	62	43	70	8.5	185
7	1	55	43	61	6.8	160
8	-1	57	38	64	7.1	168
9	1	50	41	65	7.8	170
10	-1	65	39	61	7.6	160
11	-1	61	39	60	6.8	159
12	1	70	44	71	8.8	188
13	1	52	40	66	7.6	173
14	1	72	42	68	8.9	180
15	1	67	44	70	8.4	183
16	-1	52	37	64	6.7	168
17	1	70	43	69	7.9	187

示例代码：

```
model.load_tab_data('data/Height_data_train.csv')
```

返回值包括 `x_train`，`y_train`，`x_val`，`y_val`，分别表示训练集和验证集的数据集和标签。如果数据集中没有验证集，则返回 `None`。

参数说明：

`data_path`：CSV数据集路径。

`train_val_ratio`：训练集与验证集的比例，float类型，默认值为1.0，即不划分，全部作为训练集。

`shuffle`：是否打乱数据，默认为True。

`random_seed`：随机数种子，用于复现代码效果。

注意：聚类算法通常没有明确的分类输出标签列，因此不能直接使用 `load_t`

 master ▼

方法2：使用 `load_dataset` 方法载入，指定文件类型、特征列和标签列

这种方法载入数据更加灵活，适合需要辅助做特征选择的情况。特征和标签列不需要连续。对于聚类任务，只能选择此方法载入数据，因为它支持仅载入特征列。

示例代码1：

```
# 载入数据集，并指定特征列和标签列，1-4列为特征（x），5列为标签（y）
model.load_dataset('./lenses.csv', type='csv', x_column=[1,2,3,4], y_column=[5], shuffle=False)
```

示例代码2：

```
# 载入数据集，并指定特征列和标签列，1-4列为特征（x），5和8列为标签（y）
model.load_dataset('./lenses.csv', type='csv', x_column=[1,2,3,4], y_column=[5,8], shuffle=False)
```

参数说明：

`type`：表示X和y的输入格式，可选项为'csv'、'numpy'、'pandas'、'list'、'txt'。

`x_column`：指定特征列，如[1,2,3,4]。

`y_column`：指定标签列，如[5]或者[5,6]、[5,8]等。

`split`：是否划分训练集、验证集，默认为True。

`shuffle`：是否打乱数据，让数据随机化，默认为True。

`scale`：是否对数据进行归一化，默认为False。

`show`：显示5条数据。默认为True。

(2) 针对图片类型数据

如需载入图像数据，可读取图像数据转换为Numpy数组后，直接从变量载入数据。使用上文介绍过的载入数据更加灵活的 `load_dataset` 方法载入即可。

```
# 载入数据，并说明特征列和标签列
model.load_dataset(X=train_x, y=train_y, type='numpy')
```

`x` 表示数据特征，`y` 表示标签。可再设置 `x_column` 和 `y_column` 参数，不设置 `x_column` 和 `y_column` 表示载入数据的所有列。

其他参数同上文。

使用外部工具载入数据后赋值

BaseML支持pandas和numpy的数据，赋值给 `x_train` 和 `y_train` 即可。

示例代码：

```
import pandas as pd
from BaseML import Regression as reg

# 从 CSV 文件中读取数据
data = pd.read_csv('data/accelerated_motion_dataset_train.csv')

# 分离特征和标签
x = data.iloc[:, :3].values # 前 3 列是特征
y = data.iloc[:, 3:].values # 后 2 列是标签

# 构建多项式回归模型
model = reg(algorithm='Polynomial')

# 手动设置训练数据
model.x_train = x
model.y_train = y
```

辅助工具：BaseML内置的图像处理模型 `ImageLoader`

如需对图片进行处理后再载入，可借助BaseML内置的图像处理模型 `ImageLoader`。

简单使用示例：

以读取ImageNet格式的MNIST数据集为例，进行说明。

```
from BaseML import IMGLoader
# 指定数据集路径
train_path = '/data/QX8UBM/mnist_sample/training_set'
test_path = '/data/QX8UBM/mnist_sample/test_set'
# 初始化图片加载器并载入数据集
img_set = IMGLoader.ImageLoader(train_path, test_path, size=28)
# 图像数字化处理
X_train, y_train, X_test, y_test = img_set.get_data(method='flatten')
```

```
# 载入数据，从变量载入
model.load_dataset(X=X_train, y=y_train, type='numpy')
```


3. 模型训练

```
# 模型训练
model.train()
```

4. 模型评估

模型评估可以在训练集的基础上进行评估，也可以额外制定数据集进行评估。但由于算法有一定差异，并不是所有算法都支持评估。

```
# 模型评估
model.valid(metrics='acc')
```

`valid` 方法的返回值有2个，分别是评估指标计算结果和验证集的推理结果。

参数说明：

`metrics`：评估指标选择，默认为'acc(accuracy)'，还支持precision,recall,f1,auc,r2,mse,mae、Silhouette Score、Calinski-Harabasz Score、Davies-Bouldin Score等。分类任务一般选择'acc(accuracy)'，回归任务可以选择'r2'(R平方值)或'mse'(MSE值)，聚类任务一般选择Silhouette Score（轮廓系数），评估指标的说明详见[后文](#)。

`path / x,y`：验证集的路径/验证集的特征和验证集的标签，如载入数据时设置了自动划分训练集和验证集，此时可不传入此参数，其他情况下基本需要传入，否则将报错。

注：聚类任务只需传入 `metrics`，因为聚类和分类回归不一样，没有类别标签，即y只有预测值，没有真实值，因此聚类任务验证实际是对训练集的效果评价，因此不传入x和y也可直接验证。

如载入的是文本数据且直接使用 `load_tab_data` 载入，评估时可直接载入一个数据集格式要求及输入列数量等和训练数据保持高度一致即可的验证集进行模型评估即可。

```
model.valid('data_val.csv',metrics='acc') # 载入验证数据并验证
```

除了传入验证集的路径，还可在 `valid` 方法中传入x验证集的特征和y验证集的标签进行模型评估，此方式更加灵活。

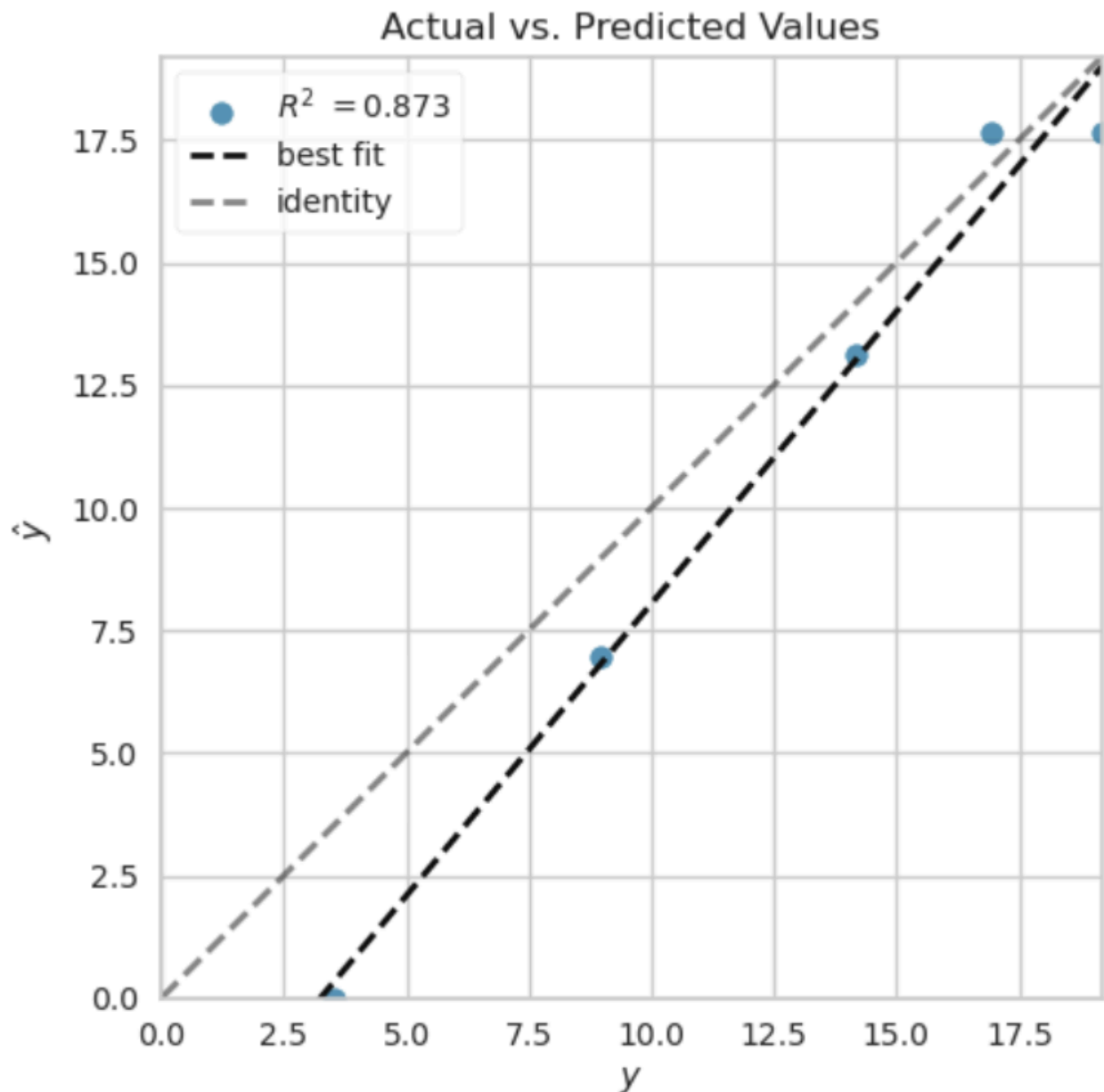
```
# 模型评估
model.valid(x=val_x,y=val_y,metrics='acc')
```

5. 评价指标可视化

```
model.metricplot()
```

使用前面运行的代码中读取的验证集的特征`val_x`和验证集的标签`val_y`进行绘制，如无或想自行传入，可自行设置。

验证`r2-score`为：87.25141807306217%



上图是线性回归任务绘制的可视化图，验证集已有的输出`y`为横坐标，通过模型推理得到的结果`ŷ`为纵坐标，如果两者构成的坐标点落在灰色虚线上，说明模型完全契合验证数据。而实际构成的点没有落在灰色虚线上，而是围绕黑色虚线分布，两条虚线相差越大，说明模型效果越差。

- 注：部分模型尚未支持。

6. 模型推理

```
# 给定一组数据，推理查看效果
y=model.inference([[1,1,1,1]])
```

- 注：由于部分算法特殊性，一些算法并不支持推理，这些算法（如 `Spectral clustering` 谱聚类）只能针对训练集的数据进行一次性的拟合。

7. 模型的保存与加载

```
# 保存模型
model.save('my_CART_model.pkl')
# 加载模型
model.load("my_CART_model.pkl")
```

参数为模型保存的路径。

模型保存后可加载模型进行模型推理，参考代码如下：

```
# 加载模型
model.load("my_CART_model.pkl")
# 给定一组数据，推理查看效果
y=model.inference(data)
```

8. 模型应用

模型应用是将训练好的模型部署到实际场景中，例如集成到网站或移动应用中。一般来说，一个训练模型的工具也会自带了推理功能，如在BaseML训练好模型并保存，下次使用时以同样的方式导入BaseML库并载入模型进行推理即可（参照前文的6.模型推理与7.模型载入）。还有种方式是借助一些通用的模型推理库，如XEdu工具的XEduHub库，支持推理各种工具训练的模型，此类库的安装一般比机器学习开发工具简单很多。也可以[借助XEduHub库完成推理](#)和应用，核心代码如下。

```
from XEdu.hub import Workflow as wf
baseml = wf(task='baseml',checkpoint='./model.pkl') # 指定使用的pkl模型
data = [[1, 1, 1, 1]]# 指定测试数据，根据训练模型时使用的数据来定
result= baseml.inference(data=data) # 进行模型推理
print(result)
```

附录

1. 分类、回归和聚类

如果预测任务是为了将观察值分类到有限的标签集合中，换句话说，就是给观察对象命名，那任务就被称为**分类**任务。另外，如果任务是为了预测一个连续的目标变量，那就被称为**回归**任务。

所谓**聚类**，即根据相似性原则，将具有较高相似度的数据对象划分至同一类簇，将具有较高相异度的数据对象划分至不同类簇。聚类与分类最大的区别在于，聚类过程为无监督过程，即待处理数据对象没有任何先验知识，而分类过程为有监督过程，即存在有先验知识的训练数据集。

2. 常见评估指标

ACC (准确率)：用于分类任务。衡量模型预测正确的实例占有所有实例的比例。想象一下你在做一个分类测试，准确率就是你回答正确的问题数量占总问题数量的比例。准确率越高，表示模型整体预测的准确性越强。

Precision (精确率)：用于分类任务。用来衡量正类预测的准确性。想象一下，你用网捞鱼，精确率就是你捞上来的所有鱼中真正你想要的鱼的比例。在分类任务中，精确率表示的是被正确预测为正类（如真实的病例、真正的信号）的实例占有所有预测为正类的实例的比例。在类分布不均衡的情况下，精确率通常比准确率更能反映模型的实际性能。

Recall (召回率)：用于分类任务。用来衡量模型捕捉正类实例的能力。召回率表示的是正确预测为正类的实例占有所有实际正类的实例的比例。

F1 Score (F1分数)：用于分类任务。平衡精确率和召回率，尤其适用于不平衡数据集。精确率和召回率像是一对矛盾的兄弟，一个高，另一个往往就低。F1分数就是这两个兄弟的和平协议，它找到了两者之间的平衡点。F1分数是精确率和召回率的调和平均数，用于给出单个度量，在两者之间取得平衡。

AUC (曲线下面积)：用于分类任务。衡量模型区分两个类别（正类和负类）的能力。想象一下你画了一条线分隔好人和坏人，AUC衡量的就是这条线分隔好坏人能力的指标。AUC是接收者操作特性（ROC）曲线下的面积，值越高，表示模型区分正类和负类的能力越强。

R2 (决定系数)：用于回归任务。衡量模型解释数据变异的能力。这个指标告诉我们模型对现实数据的拟合程度如何。值为1意味着模型完美拟合数据，值为0意味着模型不比随机猜测好。你可以把它看作是评分你的模型对数据了解程度的考试成绩。

MSE (均方误差)：用于回归任务。衡量模型预测值和实际值差异的平方的平均值。如果你射箭，每次射箭偏离靶心的距离的平方的平均值，就是均方误差。它衡量的是实际值之间差异的平方的平均值。MSE越小，表示模型预测的精确度越高。

MAE (平均绝对误差): 用于回归任务。衡量模型预测值和实际值差异的绝对值的平均值。同样的射箭比喻，每次射箭偏离靶心的距离的平均值，就是平均绝对误差。它是模型预测值与实际值之间差异的绝对值的平均值。

Silhouette Score (轮廓系数): 用于聚类任务。衡量簇的紧密性和分离度。这个指标像是给每个点打分，看它是不是跟自己组里的点挨得很近，同时跟其他组的点挨得很远。分数从-1到1，分数高说明聚类效果好。

Calinski-Harabasz Score (CH指数): 用于聚类任务。基于簇内和簇间离散度来评估聚类的质量。这个指标就像评价一个足球队的防守和进攻。防守紧凑表示同一个簇内的点很接近，进攻犀利表示不同簇的点相隔很远。CH指数越高，聚类效果越好。

Davies-Bouldin Score (DB指数): 用于聚类任务。评估簇的分离度，较低的值表示更好的分离效果。这个指标试图衡量簇内的相似性和簇间的差异。想象你有很多圈朋友，DB指数就像衡量每个圈子里的朋友有多相似，以及不同圈子之间的差异有多大。DB指数越低，聚类效果越好。

3. 其他算法

贝叶斯分类

贝叶斯分类算法常用于解决不确定问题，如人们普遍认为夜里下雨，第二天早晨草地会湿，实际到了早上草地可能就干了，也许是因为风的因素，解决这类问题往往需要根据人类已有的经验来计算某种状态出现的概率，这种方式叫做贝叶斯推理。贝叶斯分类算法是基于贝叶斯定理的一种算法，即“简单”地假设每对特征之间相互独立。

贝叶斯定理： $P(A|B)$ 表示事件B发生的条件下事件A发生的概率， $P(A|B)$ 等于事件A发生的条件下事件B发生的概率乘以事件A发生的概率 $P(A)$ ，再除以事件B发生的概率 $P(B)$ 。

```
# 实例化模型，模型名称选择NaiveBayes
model=cls('NaiveBayes')
```

k均值

k均值 (k-means) 算法是一种基于数据间距离迭代求解的聚类算法，通过分析数据之间的距离，发现数据之间的内在联系和相关性，将看似没有关联的事物聚合在一起，并将数据划分为若干个集合，方便为数据打上标签，从而进行后续的分析 and 处理。k代表划分的集合个数，means代表子集内数据对象的均值。

```
# 实例化模型，模型名称选择'Kmeans'
model = clt('Kmeans', N_CLUSTERS=5)
```



参数 `N_CLUSTERS` 表示簇数量 (K的值)，默认值为5。

谱聚类

谱聚类 (spectral clustering) 算法主要思想是把所有的数据看做空间中的点，这些点之间可以用边连接起来。将聚类问题转为图分割问题：距离较远（或者相似度较低）的两个点之间的边权重值较低，而距离较近（或者相似度较高）的两个点之间的边权重值较高，将所有数据点组成的图分割成若干个子图，让不同的子图间边权重和尽可能的低，而子图内的边权重和尽可能的高，从而达到聚类的目的。

```
# 实例化模型，模型名称选择 'SpectralClustering',  
model = clt('SpectralClustering', N_CLUSTERS=5)
```

参数 `N_CLUSTERS` 表示簇数量（子图数量），默认值为5。

Agglomerative clustering

Agglomerative clustering 是一种自底而上的层次聚类方法，它能够根据指定的相似度或距离定义计算出类之间的距离。首先将每个样本都视为一个簇，然后开始按一定规则，将相似度高的簇进行合并，直到所有的元素都归为同一类。

```
# 实例化模型，模型名称选择 'Agglomerative clustering'  
model = clt('Agglomerative clustering', N_CLUSTERS=5)
```

参数 `N_CLUSTERS` 表示簇数量，默认值为5。

Birch聚类

Birch (Balanced Iterative Reducing and Clustering using Hierarchies) 是层次聚类的典型代表，天生就是为处理超大规模数据集而设计的，它利用一个树结构来快速聚类，这个树结构类似于平衡B+树，一般将它称之为聚类特征树(Clustering Feature Tree，简称CF Tree)。这颗树的每一个节点是由若干个聚类特征(Clustering Feature，简称CF)组成。

```
# 实例化模型，模型名称选择 'Birch'  
model = clt('Birch', N_CLUSTERS=5)
```

参数 `N_CLUSTERS` 表示簇数量，默认值为5。

Develop and launch modern apps with MongoDB Atlas, a resilient data platform.